

A class implementation of Binary Search Tree in C++

9/18/2015

8 Comments

A place to practice the coding interview.

Author

Hamed Kiani

Categories

[All](#)

[Arrays And Strings](#)

[Linked List](#)

[Stack And Queue](#)

[Trees And Graphs](#)

Archives

[September 2015](#)

[July 2015](#)

[RSS Feed](#)

```

1 /*****
2 *****/
3 Chapter 4 Trees and Graphs
4
5 This is a class implementation of Binary Search Tree (BST), containing:
6
7   inser(): insert a value in a BST
8   isBalanced(): check if a BST is balanced, a BST is balanced if the difference of left
9   getHeight(): returns the height of a BST
10  deleteBST(): deletes a BST
11  inOrder():   prints a BST i in-order fashion
12  preOrder(): prints a BST i pre-order fashion
13  postOrder(): prints a BST i post-order fashion
14
15 By: Hamed Kiani (Sep. 18, 2015)
16 *****/
17 *****/
18
19 #include "stdafx.h"
20 #include <iostream>
21 using namespace std;
22
23 // tree node, including left and right pointers, data
24 struct Node{
25     Node(int value): data(value), left(NULL), right(NULL) {}
26     int data;
27     Node *left;
28     Node *right;
29 };
30
31 //////////////////////////////////////
32 // BST class
33 class BST{
34 private:
35     Node *_root;
36     void insert(Node *treeNode, int data);
37     bool isBalanced(Node *treeNode);
38     int  getHeight(Node *treeNode);
39     void deleteBST(Node *treeNode);
40     void inOrder(Node * treeNode);
41     void preOrder(Node * treeNode);
42     void postOrder(Node * treeNode);
43 public:
44
45     BST(); // constructor
46     ~BST(); // destructor
47
48     void insert(int data){ insert(_root, data);}
49
50     int getHeight(){return getHeight(_root);}
51     Node * getMaxNode();
52     Node * getMinNode();
53
54     void deleteBST() {deleteBST(_root);}
55
56     bool isBalanced(){return isBalanced(_root);    }
57
58     void inOrder() {inOrder(_root);}
59     void preOrder(){preOrder(_root);}
60     void postOrder(){postOrder(_root);}
61 };
62
63 //////////////////////////////////////
64 BST::BST()
65 {
66     _root = NULL;
67 }
68

```

```

69 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
70 void BST::insert(Node *treeNode, int data)
71 {
72     if (!treeNode)
73     {
74         treeNode = new Node(data);
75         _root = treeNode;
76     }
77     else
78     {
79         if (data < treeNode->data)
80         {
81             if (!treeNode->left)
82             {
83                 Node *treeTemp = new Node(data);
84                 treeNode->left = treeTemp;
85             }
86             else
87                 insert(treeNode->left, data);
88         }
89         else
90         {
91             if (!treeNode->right)
92             {
93                 Node *treeTemp = new Node(data);
94                 treeNode->right = treeTemp;
95             }
96             else
97                 insert(treeNode->right, data);
98         }
99     }
100 }
101
102 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
103 int BST::getHeight(Node *treeNode)
104 {
105     if (!treeNode)
106         return 0;
107     return 1 + max(getHeight(treeNode->left), getHeight(treeNode->right));
108 }
109
110 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
111 bool BST::isBalanced(Node *treeNode)
112 {
113     if (!treeNode)
114         return false;
115     int leftHeight = getHeight(treeNode->left);
116     int rightHeight = getHeight(treeNode->right);
117
118     if (abs(leftHeight - rightHeight) > 1)
119         return false;
120     return true;
121 }
122
123 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
124 Node * BST::getMaxNode()
125 {
126     if (!_root)
127     {
128         cout << " the BST is empty!" << endl;
129         return NULL;
130     }
131     Node * treeNode = _root;
132     while(treeNode->right)
133         treeNode = treeNode->right;
134     return treeNode;
135 }
136
137 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
138 Node * BST::getMinNode()
139 {
140     if (!_root)
141     {
142         cout << " the BST is empty!" << endl;
143         return NULL;
144     }
145     Node * treeNode = _root;
146     while(treeNode->left)
147         treeNode = treeNode->left;

```

```

148         return treeNode;
149     }
150
151     ///////////////////////////////////////////////////////////////////
152     void BST::deleteBST(Node *treeNode)
153     {
154         if (!treeNode)
155             return;
156
157         Node * curTreeNode = treeNode;
158         Node * leftTreeNode = treeNode->left;
159         Node * rightTreeNode = treeNode->right;
160         delete(curTreeNode);
161         deleteBST(leftTreeNode);
162         deleteBST(rightTreeNode);
163     }
164
165     ///////////////////////////////////////////////////////////////////
166     BST::~BST()
167     {
168         deleteBST();
169     }
170
171     ///////////////////////////////////////////////////////////////////
172     void BST::inOrder(Node * treeNode)
173     {
174         if (!treeNode)
175             return;
176         inOrder(treeNode->left);
177         cout << treeNode->data << " ";
178         inOrder(treeNode->right);
179     }
180
181     ///////////////////////////////////////////////////////////////////
182     void BST::preOrder(Node * treeNode)
183     {
184         if (!treeNode)
185             return;
186         cout << treeNode->data << " ";
187         preOrder(treeNode->left);
188         preOrder(treeNode->right);
189     }
190
191     ///////////////////////////////////////////////////////////////////
192     void BST::postOrder(Node * treeNode)
193     {
194         if (!treeNode)
195             return;
196         postOrder(treeNode->left);
197         postOrder(treeNode->right);
198         cout << treeNode->data << " ";
199     }
200
201     ///////////////////////////////////////////////////////////////////
202     ///////////////////////////////////////////////////////////////////
203     int _tmain(int argc, _TCHAR* argv[])
204     {
205         BST myBST;
206         myBST.insert(5);
207         myBST.insert(10);
208         myBST.insert(3);
209
210         myBST.insert(51);
211         myBST.insert(110);
212         myBST.insert(13);
213
214         int h = myBST.getHeight();
215         cout << "the height of this Bst is : " << h << endl;
216
217         Node * mx = myBST.getMaxNode();
218         cout << "max value: " << mx->data << endl;
219
220         Node * mi = myBST.getMinNode();
221         cout << "min value: " << mi->data << endl;
222
223         bool isbal = myBST.isBalanced();
224         if (isbal)
225             cout << "BST is balanced! " << endl;
226         else

```

```
227         cout << "BST is not balanced! " << endl;
228
229         cout << " in-order traverse is : " << endl;
230         myBST.inOrder();cout << endl;
231         cout << " pre-order traverse is : " << endl;
232         myBST.preOrder();cout << endl;
233         cout << " post-order traverse is : " << endl;
234         myBST.postOrder();cout << endl;
235 }
```

8 Comments

Muhammad Umer Farooq

11/12/2018 12:47:53 am

What happen if insert number is equal to previous number enter in tree??

Reply

Bhavesh Pawar

1/7/2020 12:40:41 am

then code becomes more complicated so right now we need to consider that user gives us unique data

Reply

Anand Mishra

1/7/2020 12:47:33 am

keep your fucking mouth off dont interfair in anothers fucking life you bc

Bhavesh Pawar

1/7/2020 12:39:10 am

please give codes which are understandable to the user

Reply

Soham Kamthe

1/7/2020 12:42:35 am

if you are getting then get us some easy codes in yopur own laguage

Reply

Maria Chase

12/23/2020 09:06:27 pm

I enjoyeed reading your post

Reply

Brij Bhushan

4/7/2021 12:14:42 am

Indeed I found this article more informative, thanks for sharing this article.

Reply

John Cole

8/28/2023 11:38:24 am

Thanks for sharing !!

Reply

Leave a Reply.

Copyright © 2020, Hamed Kiani